

The Agent Architecture Landscape

Five patterns you will meet in real production systems — from single-loop agents to A2A collaboration.

PATTERN 01

Single vs Multi-Agent



Solo Crew

One reasoning loop vs a coordinated team. Team scales expertise but adds orchestration overhead.

PATTERN 02

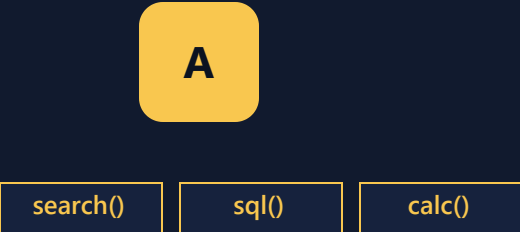
RAG-based Systems



Ground answers in your own corpus. Retrieval before generation cuts hallucination.

PATTERN 03

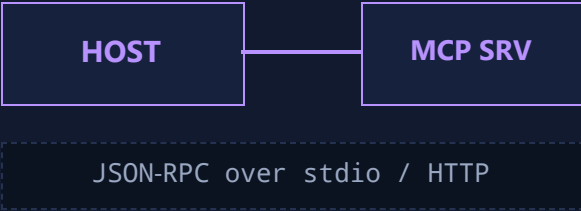
Tool-Augmented Agents



LLM chooses external functions — APIs, DBs, code — to act on the world.

PATTERN 04

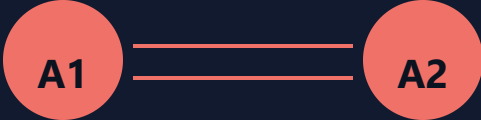
MCP-based Architecture



Standard wire protocol so any host can plug into any tool / data server.

PATTERN 05

A2A Collaboration



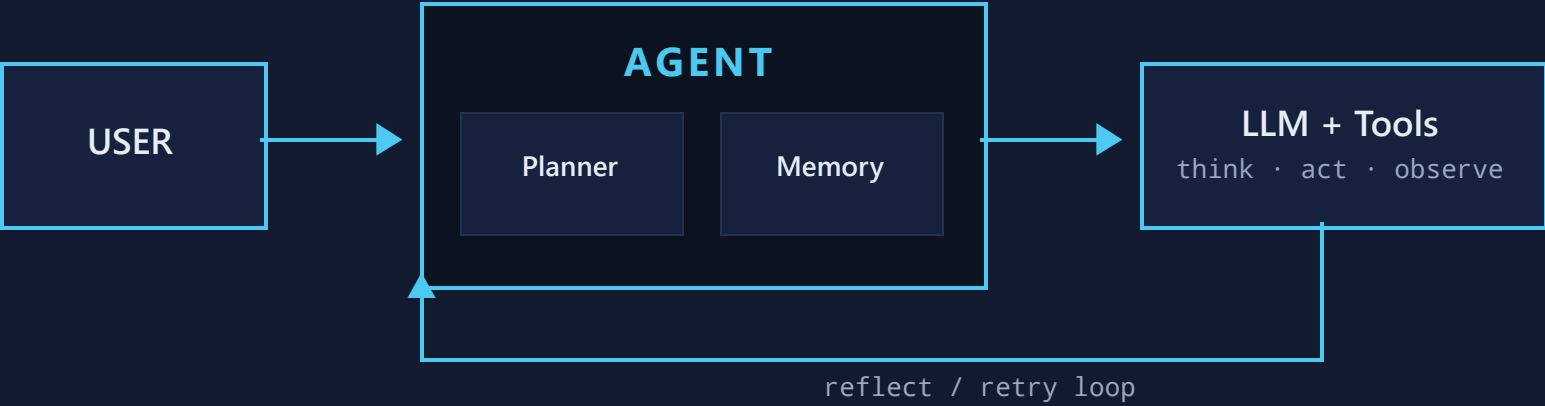
Peer agents negotiate, delegate, and hand off tasks over an open protocol.

Single Agent vs Multi-Agent

Same job, two block diagrams. Pick by task decomposability, not by hype.

SINGLE AGENT

One planner, one memory, one action loop.

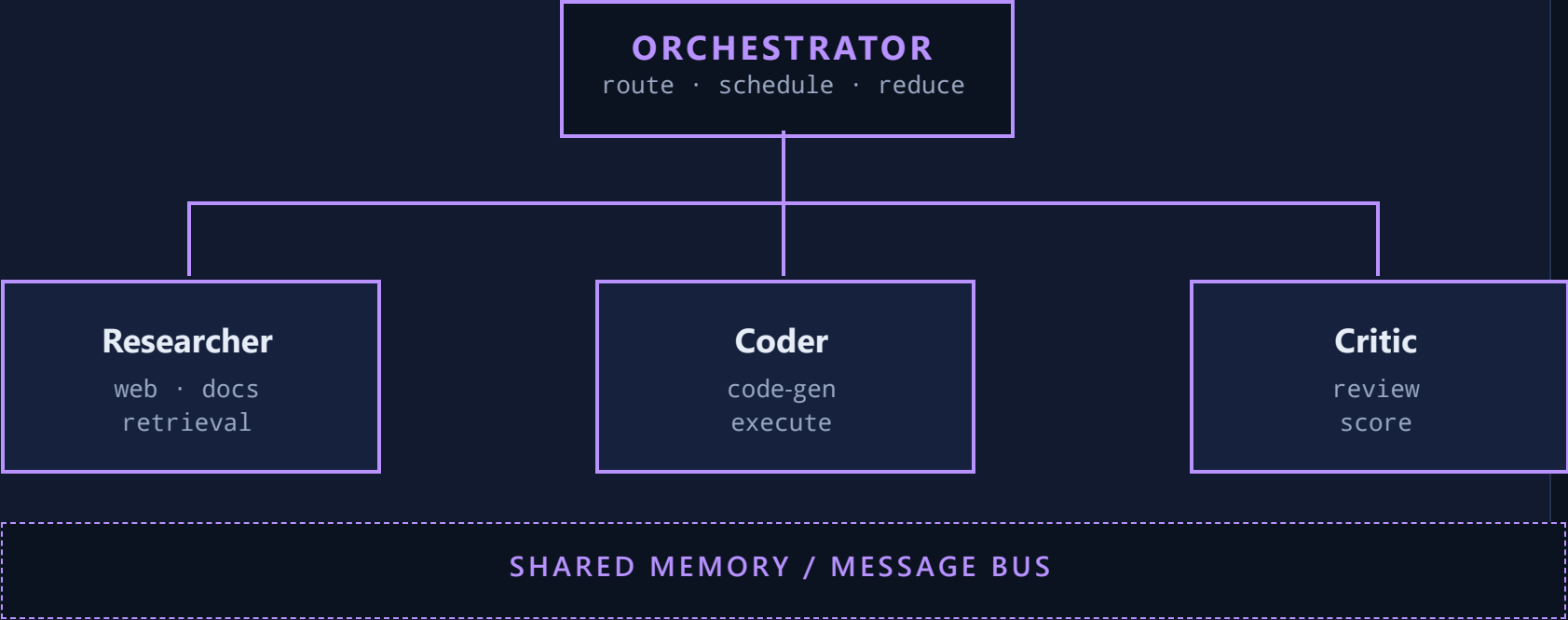


WHEN TO USE

- Task fits inside one context window
- Sequential reasoning, few branches
- Simpler ops, cheaper tokens, easier to debug
- Latency budget under ~5 seconds

MULTI-AGENT

Orchestrator + specialised workers with shared blackboard.

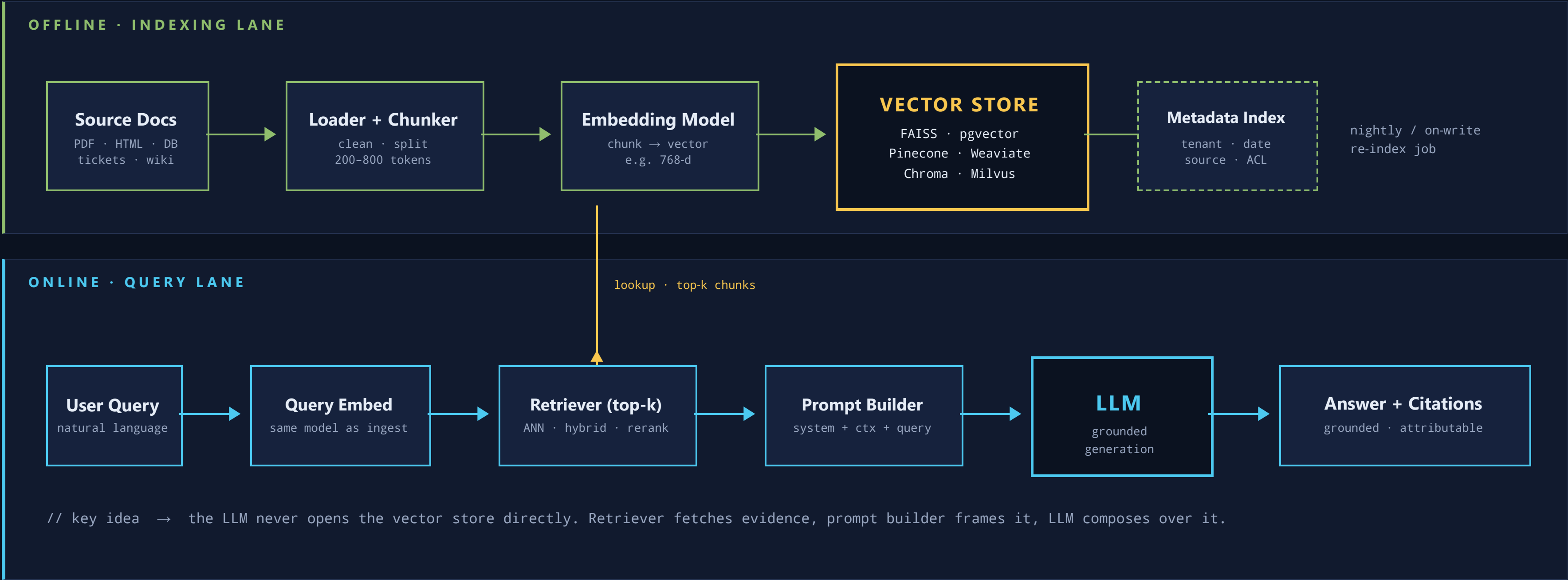


WHEN TO USE

- Task decomposes into distinct skills / roles
- Long horizons, parallelisable subtasks
- Needs review / adversarial checks
- Accept higher token cost + orchestration risk

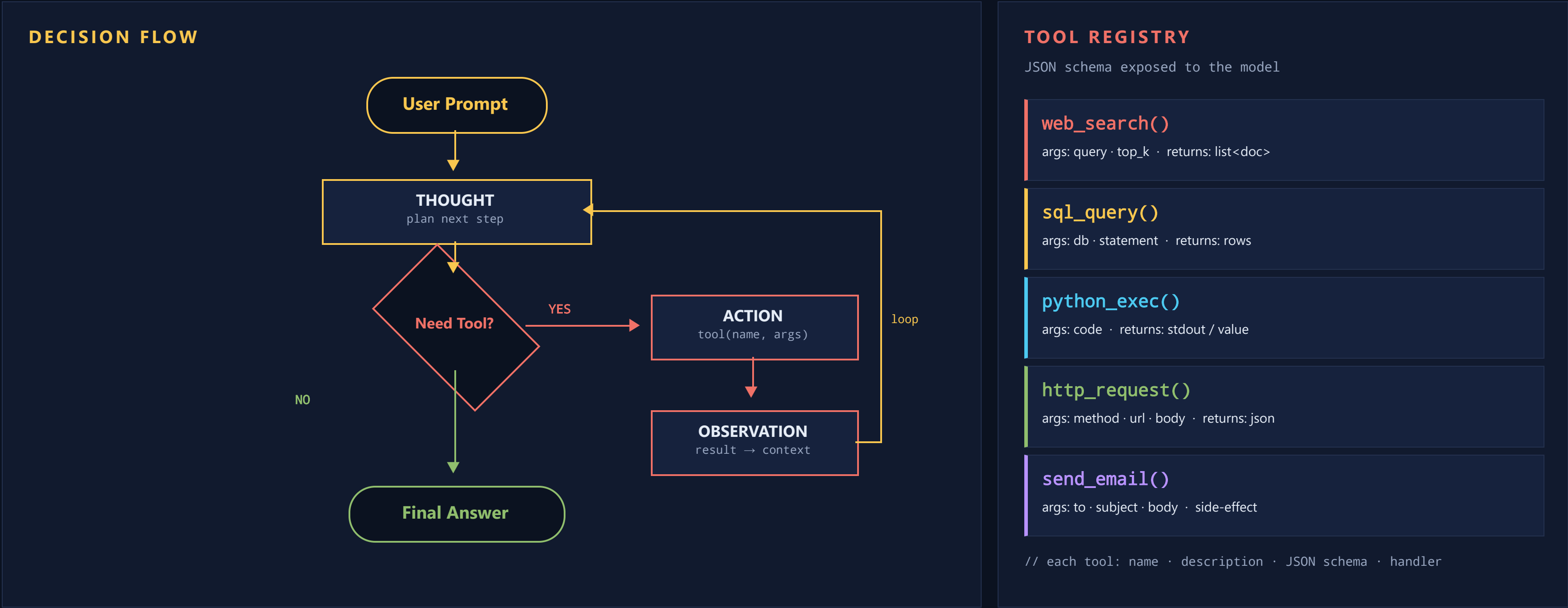
RAG-Based System · End-to-End Wiring

Two lanes: an offline indexing lane feeds an online query lane through a vector store.



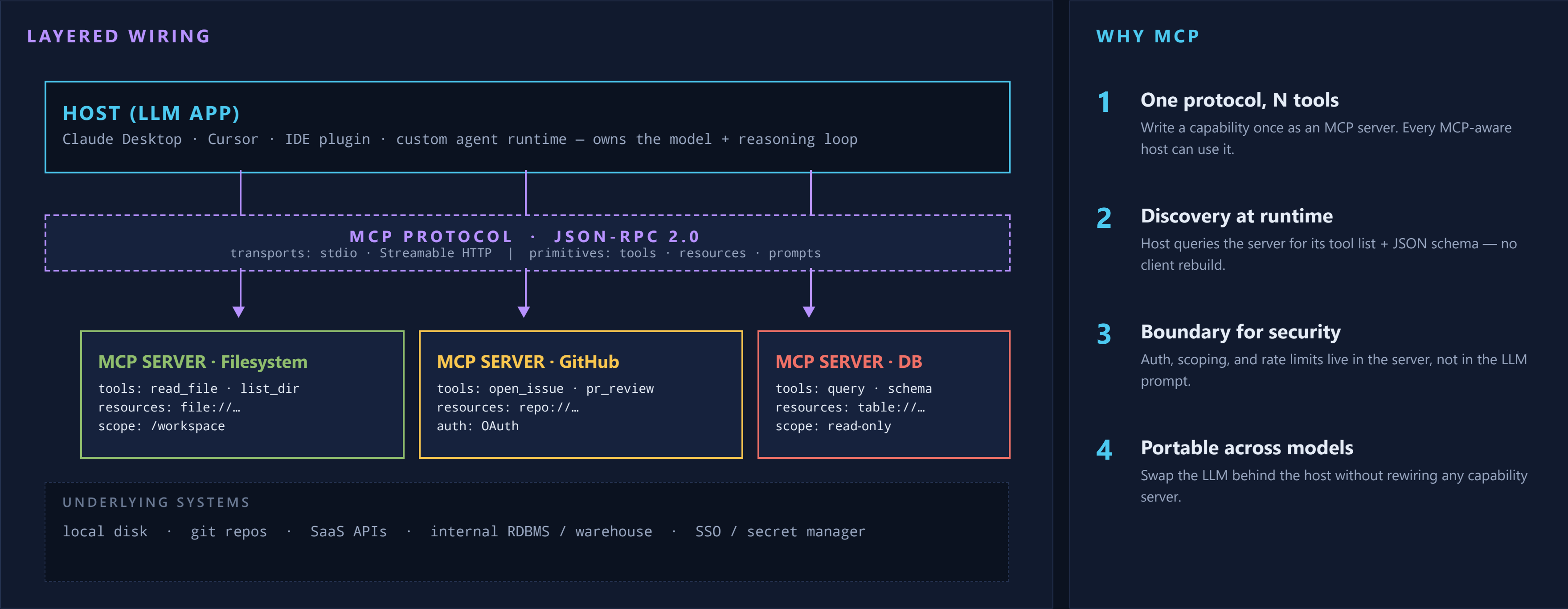
Tool-Augmented Agent · ReAct Loop

The LLM decides whether to call a tool, observes the result, and iterates until it has an answer.



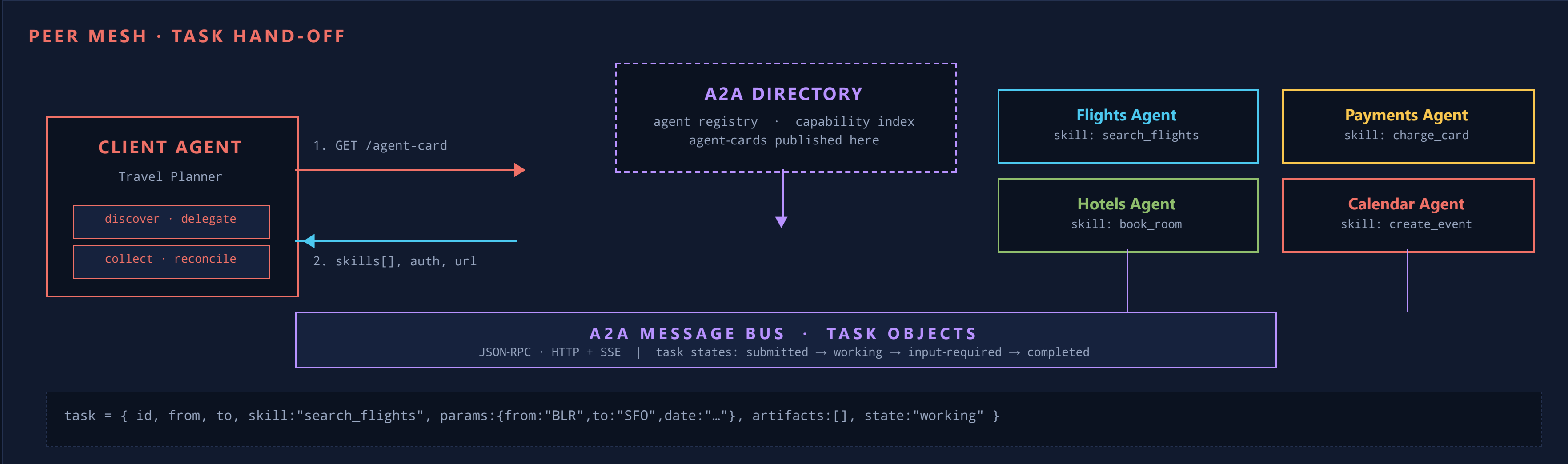
MCP-Based Architecture

Model Context Protocol — a USB-C for LLMs. One host, many pluggable capability servers.



A2A · Agent-to-Agent Collaboration

Peer agents publish capabilities, negotiate tasks, and hand off work over an open messaging protocol.



Interoperability

Agents built by different teams / vendors talk without shared codebase.

Long-running tasks

Streaming updates + input-required states — not just request / reply.

Opaque peers

A remote agent's model, prompt, and tools stay private — only its skills are exposed.

Ingestion → Embeddings → Retrieval → Generation

Four stages. Each one has its own failure modes — learn them before you tune anything.

01 · INGESTION

RAW → CLEAN CHUNKS

DO

- Extract text from PDF · HTML · Docs
- Strip boilerplate, headers, footers
- Chunk 200–800 tokens, ~10% overlap
- Attach metadata (source, section, ACL)

FAILURE MODE

Bad chunks = bad retrieval. Splitting mid-sentence or across sections destroys context.

02 · EMBEDDINGS

TEXT → VECTORS

"chunk text"

[0.12, -0.83, 0.44, ... 768d]

text-embedding-3-large · bge-m3

e5-mistral · voyage-2

DO

- Pick one model, use it for both sides
- Normalise vectors (L2) for cosine sim
- Store vector + chunk + metadata
- Version the model — re-embed on change

FAILURE MODE

Query and index using different models = geometry mismatch, similarity is noise.

03 · RETRIEVAL

QUERY → TOP-K CONTEXT

DO

- ANN search, top-k = 5-20
- Hybrid: dense + BM25 keyword
- Filter by metadata (tenant, date, ACL)
- Rerank with cross-encoder

FAILURE MODE

k too small = miss evidence. k too large = LLM drowns in noise → wrong answer.

04 · GENERATION

CONTEXT + LLM → ANSWER

SYSTEM: answer only from context.
CONTEXT:
[1] chunk_a ...
[2] chunk_b ...
USER: <question>
ANSWER: ... [1][2]

DO

- Instruct model to cite chunk IDs
- Refuse when context insufficient
- Low temperature for factual Q&A
- Post-check: every claim traces to a chunk

FAILURE MODE

Without a "cite or refuse" instruction, the model backfills gaps from training memory.

Why RAG in the Enterprise

Compare RAG head-to-head with the two alternatives — pretraining and fine-tuning — on the axes that matter.

CRITERION	Pretraining	Fine-tuning	RAG
Freshness of data	Frozen at cut-off	Frozen at retrain	Live — re-index on write
Cost to add knowledge	\$\$\$\$\$ (millions)	\$\$\$ (GPU hours)	\$ (ingest + storage)
Access control	None (baked in)	None (baked in)	Row-level at retrieval
Citations / auditability	No	No	Yes — chunk IDs
Best for	General reasoning	Style / format	Facts, policies, docs

01 Fresh knowledge

Answers reflect today's policy, ticket, price — not last quarter's snapshot.

02 Grounded answers

Every claim links back to a source chunk — auditable, legally defensible.

03 Data stays yours

Corpus lives in your vector store; no training data leaves the boundary.

04 Least-privilege

Filter chunks by user role at query time — same LLM, per-user context.

Where RAG Actually Breaks

Six pitfalls you will hit in production — each with its symptom and its fix.

PITFALL

01

Chunk boundaries destroy meaning

Fixed-length splits cut through tables, code blocks, and clause groups — key facts land in different chunks.

FIX

Semantic / structural chunking. Preserve tables + code as single units. Overlap 10–15%.

PITFALL

02

Retriever fetches near-duplicates

Top-k comes back full of paraphrased copies. Context window burns on redundancy, real evidence gets truncated.

FIX

MMR (max-marginal-relevance) rerank. Deduplicate by hash. Cluster-then-pick.

PITFALL

03

Stale index

Docs get updated in the source system but the vector store still serves last week's version — confidently.

FIX

CDC / event-driven re-index. Track doc hash + version in metadata.

PITFALL

04

Hallucination on empty context

When retrieval returns nothing relevant, the LLM confidently invents from memory.

FIX

Score threshold + refuse template: "I don't have that information."

PITFALL

05

ACL leakage across tenants

One shared index, no per-query filter — user A sees fragments from user B's private docs.

FIX

Metadata filter on every query. Per-tenant namespaces if strict isolation.

PITFALL

06

Prompt injection via docs

A retrieved chunk contains "ignore previous instructions..." — the LLM obeys.

FIX

Wrap context as untrusted data. Instruction-hierarchy prompts. Output filters.

Document Q&A Agent · Reference Wiring

Vector search · simple retrieval · response grounding — end-to-end block diagram + build steps.

